

MANUEL ANDROID

Manuel d'utilisation — IRON MAN AI : Analyse Android

Le module **Android** de IRON MAN AI réalise une analyse statique de sécurité d'un fichier APK/AAB (application Android) : permissions, composants exportés, secrets codés en dur, code à risque (WebView, crypto faible, SQL, TLS...), URLs, le tout sans décompiler entièrement l'application.

 **Sécurité et éthique.** N'analysez que des applications que vous

avez développées, que vous possédez, ou pour lesquelles vous avez

une **autorisation écrite** (audit interne, test d'intrusion mandaté).

Le flag `--authorized` est obligatoire. Analyser une application que

vous ne possédez pas (pour voler des secrets ou contourner des

protections) est illégal et n'est pas couvert par cet outil.

1. Prérequis

- **Python 3.10+** (le cœur est en stdlib, aucune dépendance).
- L'analyse de base (manifeste binaire, chaînes dex, secrets) fonctionne **sans aucun outil externe** : le manifeste Android est un format binaire (AXML) décodé nativement par le module.
- **Optionnel** (analyse enrichie) : - `apktool` : décode le manifeste et les ressources (`sudo apt-get install -y apktool`) - `jadx` : décompilation complète du code Java (`sudo apt-get install -y jadx`)

Vérifiez les outils disponibles :

```
cd /chemin/codescan
python mobile_scan.py --android --check
```

2. Première analyse : un APK en une commande

```
python mobile_scan.py --android --apk /chemin/app.apk --authorized
```

Le module affiche d'abord les outils disponibles, puis le **résumé** de l'analyse :

```
Package           : com.example.app
Version           : 2.4.1 (code 241)
minSdk/targetSdk  : 23 / 34
SHA-256           : 3f2a9c1d...

Total findings    : 12
  high           4  █
  medium         6  ███
  low            2  █
Permissions dangereuses : 3
Composants exportés    : 2
Secrets détectés       : 1
```

Puis le détail des findings, classés par sévérité.

3. Ce que l'outil détecte

Manifeste (AndroidManifest.xml)

- **Permissions dangereuses** : CAMERA, RECORD_AUDIO, LOCALISATION, SMS, contacts... (protection level *dangerous*).
- **Permissions à haut risque** : QUERY_ALL_PACKAGES, MANAGE_EXTERNAL_STORAGE, SYSTEM_ALERT_WINDOW, REQUEST_INSTALL_PACKAGES...
- **Composants exportés** : activity/service/receiver/provider invocables par d'autres applications.

- **Debuggable** (`android:debuggable="true"`), **backup autorisé** (`android:allowBackup="true"`), **trafic en clair** (`android:usesCleartextTraffic="true"`).
- **minSdkVersion obsolète** (< 21, Android 5.0).

Bytecode (classes*.dex)

- **Secrets codés en dur** : clés AWS (AKIA...), clés API Google (Alza...), Stripe, tokens GitHub/Slack/Firebase, clés privées PEM, `password=...` / `api_key=...` dans le code.
- **Code à risque** : WebView avec JavaScript, `addJavascriptInterface`, hachage MD5/SHA-1, chiffrement faible (DES/RC4/AES-ECB), requêtes SQL concaténées, validation TLS désactivée, accès aux identifiants de l'appareil (IMEI...), chargement dynamique de code.
- **URLs** : endpoints HTTP(S) embarqués, signalés s'ils utilisent HTTP en clair.

4. Produire un rapport HTML/JSON

```
python mobile_scan.py --android --apk app.apk --authorized -o /tmp/rapport_app  
# → /tmp/rapport_app.json + /tmp/rapport_app.html
```

- `-o rapport.html` : HTML seul ; `-o rapport.json` : JSON seul.
- Le rapport HTML est un document autonome et lisible dans un navigateur (findings triés par sévérité, permissions, composants, secrets, URLs).

5. Options utiles

Option	Effet
<code>--no-secrets</code>	Désactive la détection de secrets
<code>--no-code</code>	Désactive l'analyse du bytecode (dex)
<code>--jadx</code>	Enrichit l'analyse par décompilation jadx (lent : ~10 min sur une grosse app)
<code>-v</code> / <code>--verbose</code>	Affiche le détail complet des findings
<code>--dry-run</code>	Affiche les étapes sans rien analyser

6. Aller plus loin avec apktool / jadx

Si `apktool` est installé, le manifeste binaire est décodé par apktool

(analyse plus fidèle des ressources). Si `jadx` est installé, ajoutez

`--jadx` pour lancer une **décompilation complète** en complément :

les mêmes patterns de code à risque sont recherchés dans le code Java décompilé, avec le fichier et la ligne exacts.

⚠ La décompilation jadx d'une application réelle (milliers de classes)

prend ~**10 minutes** et consomme beaucoup de RAM. Elle est **opt-in** :

sans `--jadx`, l'analyse (manifeste + dex + secrets) prend quelques

secondes.

Installation :

```
sudo apt-get update
sudo apt-get install -y apktool jadx
```

7. Interpréter les résultats

- **high / critical** : à corriger en priorité — secret exposé, trafic en clair, composant exporté sensible, TLS désactivé.
- **medium** : à corriger selon le contexte — hachage faible, backup autorisé, minSdk obsolète.
- **low** : bonnes pratiques — permissions dangereuses justifiées, composants exportés intentionnels.

Chaque finding contient une **recommandation** précise (utiliser le

Android Keystore, passer en AES-GCM, épingler les certificats,

paramétrer les requêtes SQL...).

8. Bonnes pratiques pour une utilisation efficace

1. **Cadre légal d'abord** : app possédée ou autorisation écrite.
1. **Comparez les versions** : analysez l'APK de production ET un build de dev — les secrets et le flag debuggable s'y glissent souvent.
1. **Vérifiez les faux positifs** : un mot de passe dans un exemple de documentation embarqué n'est pas forcément un secret réel — confirmez dans le code décompilé.
1. **Intégrez à la CI** : lancez l'analyse à chaque build pour détecter les secrets avant la mise en production.
1. **Ne stockez jamais de secrets** : API keys dans le code = exposées (l'APK est téléchargeable par n'importe qui). Utilisez le backend et le Android Keystore.

9. Dépannage

Problème	Cause probable	Solution
Fichier introuvable	mauvais chemin	Vérifiez le chemin de l'APK (<code>ls -la</code>)
<code>AndroidManifest.xml</code> non lisible	APK corrompu ou non-Android	Vérifiez avec <code>file app.apk</code> (doit être un zip)
Aucun finding	app bien sécurisée... ou analyse incomplète	Installez apktool/jadx pour l'analyse approfondie
<code>--apk</code> requis	mode Android sans fichier	Ajoutez <code>--apk <fichier.apk></code>

Généré par CodeScan — manuel imprimable.